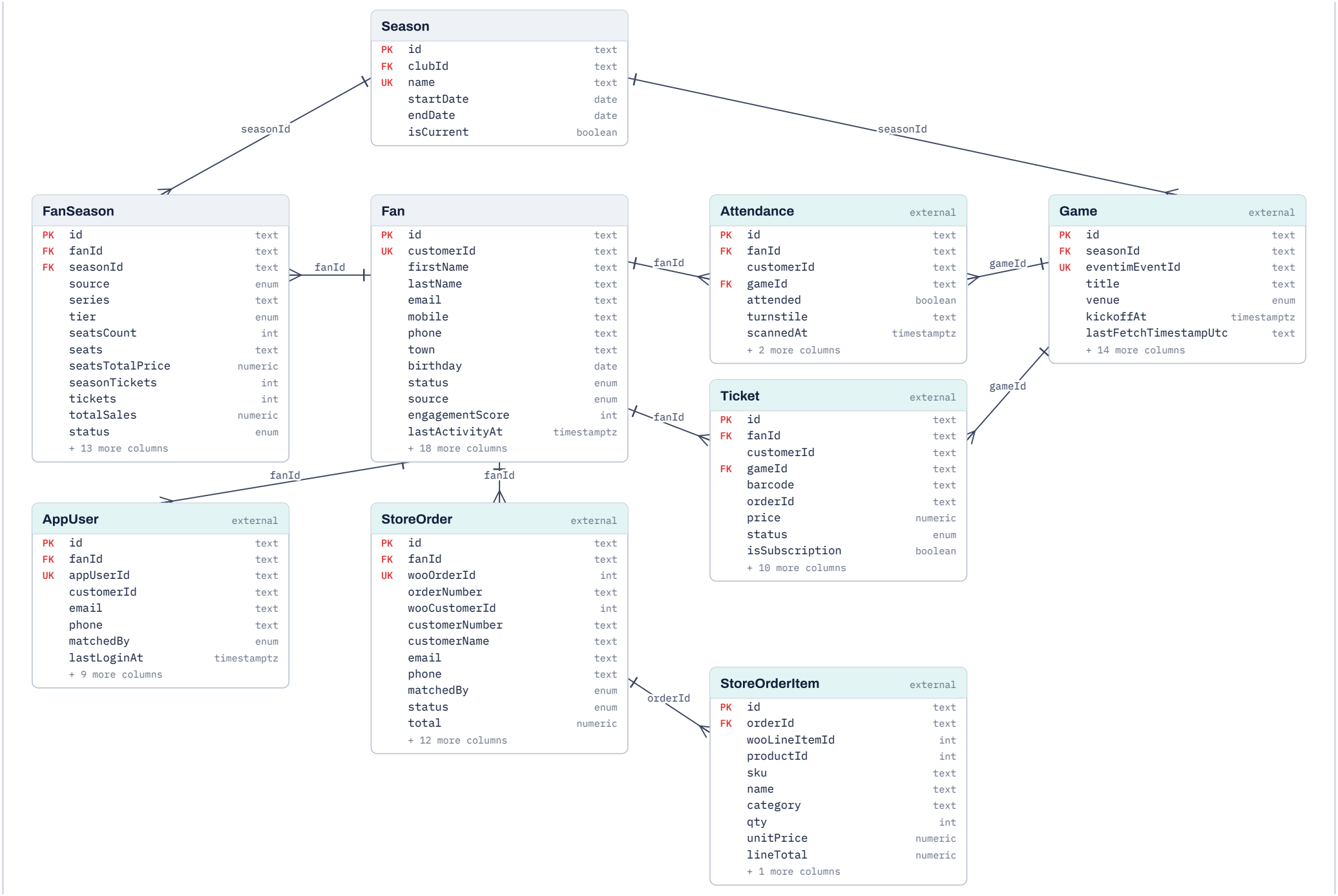
HAPOEL BE'ER SHEVA · FAN-CLUB CRM · DATABASE

Fan CRM Data Model

9 tables. **Fan** and **FanSeason** are the merged Excel. Games and tickets are pulled from Eventim Inhouse, attendance from Fortress, and WooCommerce orders and app accounts are matched by **customerId** (Eventim `CustNr`), email or phone.

ours, from the Excel copied from Eventim / Fortress / WooCommerce / app PK primary key · FK link to another table · UK unique


Season
PK id text
FK clubId text
UK name text
startDate date
endDate date
isCurrent boolean
seasonId
seasonId
FanSeason
PK id text
FK fanId text
FK seasonId text
source enum
series text
tier enum
seatsCount int
seats text
seatsTotalPrice numeric
seasonTickets int
tickets int
totalSales numeric
status enum
+ 13 more columns
fanId
Fan
PK id text
UK customerId text
firstName text
lastName text
email text
mobile text
phone text
town text
birthday date
status enum
source enum
engagementScore int
lastActivityAt timestamptz
+ 18 more columns
fanId
fanId
Attendance external
PK id text
FK fanId text
customerId text
FK gameId text
attended boolean
turnstile text
scannedAt timestamptz
+ 2 more columns
gameId
Game external
PK id text
FK seasonId text
UK eventimEventId text
title text
venue enum
kickoffAt timestamptz
lastFetchTimestampUtc text
+ 14 more columns
fanId
Ticket external
PK id text
FK fanId text
customerId text
FK gameId text
barcode text
orderId text
price numeric
status enum
isSubscription boolean
+ 10 more columns
gameId
fanId
fanId
AppUser external
PK id text
FK fanId text
UK appUserId text
customerId text
email text
phone text
matchedBy enum
lastLoginAt timestamptz
+ 9 more columns
StoreOrder external
PK id text
FK fanId text
UK wooOrderId int
orderNumber text
wooCustomerId int
customerNumber text
customerName text
email text
phone text
matchedBy enum
status enum
total numeric
+ 12 more columns
orderId
StoreOrderItem external
PK id text
FK orderId text
wooLineItemId int
productId int
sku text
name text
category text
qty int
unitPrice numeric
lineTotal numeric
+ 1 more columns

Each line is a link: the crow's foot is the "many" side, the label is the column that holds the link. Only the main columns are drawn; the full lists are at the bottom.

What each table is

TABLE	ONE ROW PER	HOLDS	COMES FROM	KEY
Fan	customer	name, email, mobile, phone, address, birthday, old CRM id, consent switches, engagement score, first seen / last activity / last app login, where the row came from	Excel	`customerId`
Season	season	2024/25 · 2025/26 · 2026/27, which one is current	Excel "Season"	`name`
FanSeason	customer × season	the merged Excel row: series and its tier (regular / silver / VIP), seats, blocks, prices, orders, pay type, years, season tickets, tickets, total sales, and a computed status (subscriber / new / returned / churned)	Excel	`fanId + seasonId`
Game	Eventim event	title, date, venue, home/away, competition, the Fortress product code and the last-fetch timestamp used as the sync cursor	Eventim SearchEvents	`eventimEventId`

TABLE	ONE ROW PER	HOLDS	COMES FROM	KEY
`Ticket`	Eventim ticket (one barcode)	customer no., order, row, seat, area, price, sales type, status, booking date, and whether it is a season-ticket seat	Eventim FetchTicketData	`gameId + barcode`
`Attendance`	customer × game	scanned in or not, turnstile and time; empty until the Fortress scan log is connected	Fortress scan log	`fanId + gameId`
`StoreOrder`	WooCommerce order	order number, dates, status, totals, payment method, channel, the billing name / email / phone and the customer number it was matched with	WooCommerce /orders	`wooOrderId`
`StoreOrderItem`	order line	product and variation id, sku, name, category, quantity, unit price, line total — what the store dashboard groups by	WooCommerce line_items + /products	`orderId + wooLineItemId`
`AppUser`	app account	last login, logins, sessions and average session length in 30 days, push token, and the customerId / email / phone it was matched with	app	`appUserId`

How the outside systems link to a fan

customerId is the Excel "Customer no." = Eventim `CustNr` = the number after the club prefix in Fortress member ids (`SHF_12345`). The club prefix in Eventim / Fortress is **SHF**. It is on every fan and never changes.

- **Eventim Inhouse (EvaXML):** nightly, `SearchEvents` for the season window, then per game `FetchTicketData` from the game's stored timestamp. Every ticket carries `CustNr`, so rows attach to the fan directly. Season-ticket seats show up on every home game with `SubscriptionFlag`.
- **Fortress:** the scan log per game (barcode, turnstile, time) fills `Attendance`. That endpoint is not integrated anywhere yet; until it is, attendance stays unknown rather than assumed. `FullActiveExtract` on the member product gives card numbers per customer.
- **WooCommerce:** every order carries `billing.email`, `billing.phone` and a name; registered buyers also carry the WordPress `customer_id`, guests carry 0. WooCommerce has no customer number, so the club adds one optional field ("מספר לקוח") to the account / checkout, stored as order meta (`_club_customer_number`). We match **customer number** → **email** → **phone**, first hit wins, and write which one matched (`matchedBy`). No hit: the order is kept with an empty `fanId` and appears in an "unmatched" list to link by hand; a later order from the same email links the earlier ones too.
- **App:** same rule, **customerId** → **email** → **phone**, one `AppUser` row per account.
- **Loading the Excel:** one merged row = one `Fan` upsert (by Customer no.) + one `FanSeason` row for that season. Column names in `FanSeason` are the Excel's. Load the three seasons oldest first; the status column is then computed against the season before.

What every screen reads

Every number and column the admin app shows today (on mock data) mapped to the tables above. "Computed" means a query or a nightly job, not a stored value.

SCREEN	SHOWS	READS FROM	NOTE
Dashboard	fans, active subscribers, not renewed, active in app, active campaigns + deltas	Fan · FanSeason.status · AppUser · Campaign	deltas compare with last month (Fan.createdAt) and last season (FanSeason)

SCREEN	SHOWS	READS FROM	NOTE
Dashboard	fan growth by month	Fan.firstSeenAt	history before the CRM launch comes from the earliest season seat / ticket per fan
Dashboard	subscriber split · recent campaigns · recent activity · store summary	FanSeason.status · Campaign · ActivityEvent · StoreOrder + StoreOrderItem	ActivityEvent rows are written by imports, campaign runs, the game sync and the score job
Fans	table columns, Excel-style filters, export	Fan · FanSeason (tier, seats of the current season)	seat column = FanSeason.seats; "imported" badge = Fan.importedAt
Fan profile	details, score, consents, app activity, tags	Fan · AppUser · ConsentLog · Tag	"member since" = Fan.firstSeenAt
Fan profile	subscription card	FanSeason (tier, seats, price, pay type, seat status)	"payment status" (paid / pending / overdue) exists in no source; the card shows pay type and seat status instead
Fan profile	attendance totals, match history, purchases	Attendance · Game · StoreOrder + StoreOrderItem	attended = null renders as "unknown" until Fortress scans arrive
Subscriptions	KPIs, subscribers per season, renewal trend by month	FanSeason per Season · FanSeason.lastSeasonTicketAt	renewal % = status vs. previous season; the monthly curve uses the season-ticket purchase date
Subscriptions	churn table (years, last payment, priority, suggested action)	FanSeason.status = churned · numberOfYears · lastSeasonTicketAt · Fan	priority and suggested action are rules over score and years, computed on read

SCREEN	SHOWS	READS FROM	NOTE
`Home / away games`	game table: subscribers, tickets, total, capacity, attended, revenue, status	Game · Ticket (isSubscription split) · Attendance	revenue = sum of Ticket.price; capacity on Game
`Home / away games`	KPIs: avg crowd, subscriber utilization, arrival %	Attendance · Ticket	need Fortress scans; away games are scanned by the host stadium, so away attendance may stay "tickets sold" only
`Home / away games`	top fans, buyer split, potential fans	Attendance · Ticket · AppUser · Fan.engagementScore · FanSeason.tier	preferred channel = the fan's first enabled opt-in
`Behavior`	segment / venue / period filters, KPIs, fan table	FanSeason · Attendance · Game (venue, kickoffAt) · AppUser	periods are date ranges over Game.kickoffAt; "last game" = latest played game per venue
`Behavior`	map and pie by city	Fan.town + a static city → coordinates lookup	the lookup is config (name, lat, lng), not fan data
`Comparisons`	seasons / games / opponents × attendance, tickets, revenue, subscribers, occupancy	Game · Ticket · Attendance · FanSeason	occupancy = attendance ÷ Game.capacity; opponent is the admin-set Game.opponent
`Comparisons`	wins / draws / losses per opponent	Game.goalsFor · goalsAgainst	Eventim has no results; entered by an admin or a results feed
`Store`	revenue, orders, avg order, top product, monthly, categories, recent orders	StoreOrder · StoreOrderItem	revenue = itemsTotal of PAID orders minus refunds; unmatched orders show the billing name

SCREEN	SHOWS	READS FROM	NOTE
Campaigns	audience filters (segment, cities, tier, score, app users, birthday / missed games / inactive triggers), results, messages	Fan · FanSeason.tier · Attendance · AppUser · Campaign · MessageLog	the tier filter moves from Subscription.type to FanSeason.tier
Settings · onboarding	club, branding, admins, language	Club · AdminUser	unchanged

Nightly sync

1. **Games:** Eventim `SearchEvents` for the season dates → upsert `Game` by `eventimEventId`. Home / away comes from `VenueShortCode` (Turner = home); competition and opponent are set once by an admin.
2. **Tickets:** per game, `FetchTicketData` with the stored timestamp → upsert `Ticket` by (game, barcode), then save the returned `NewTimestampUTC` on the game. Status V = valid, S = cancelled; rows with sales channel 6000 are skipped, as the current middleware does.
3. **Attendance:** per played game, Fortress scan log → upsert `Attendance` by (fan, game). Member barcodes change their first digit on reprint, so match member rows by customer no. + row + seat, not by barcode.
4. **Store:** WooCommerce `GET /wc/v3/orders?status=any&modified_after=<cursor>&per_page=100`, page by page → upsert `StoreOrder` by `wooOrderId` and its lines by `wooLineItemId`; the cursor is the newest `modifiedAt` we hold, so refunds and cancellations come through as updates. Line items carry no category, so `/products/{id}` is looked up once per product (cached) to fill `category`. Optional: the `order.created` / `order.updated` webhooks (HMAC-SHA256 signed) push the same upsert during the day.
5. **Aggregates:** recompute `Game.ticketsSold` / `attendance` and the fan's engagement score. The club's own points rule fits here: 50 points for the first membership season, 10

per further season, and each game 5 points for members, 1 for others.

6. **Contacts on demand:** the Eventim customer endpoint (`GetCustomerDataInhouseInt?CustomerNo=…`) refreshes one fan's email and phones when an admin asks. One call per customer, so not a nightly job.

Credentials for Eventim (username / password per club), Fortress (basic auth + app id + API key per stadium) and WooCommerce (a read-only consumer key / secret from WooCommerce → Settings → Advanced → REST API, plus the webhook secret) live in a secret store, never in the database. The Eventim customer endpoint is on a non-standard port and is expected to be IP-whitelisted; WooCommerce must be called over HTTPS.

Club, AdminUser, Tag, Campaign, MessageLog, ConsentLog, ImportJob and AuditLog already exist and stay as they are. Subscription is replaced by FanSeason; Shuttle is dropped. ActivityEvent stays: it is the dashboard's recent-activity list, written by imports, campaign runs, the game sync and the score job.

Against today's Prisma schema, Fan keeps `legacyId` and `lastActivityAt`, renames `memberSince → firstSeenAt`, and gains `source` and `enrichedAt`; Game gains `capacity`, `goalsFor`, `goalsAgainst`; StoreOrderItem is new.

All columns

Fan		31 columns	–
id	text	PK	
clubId	text	FK → Club	
customerId	text	UK	Excel "Customer no." — the ticketing customer number
firstName	text		Excel
lastName	text		Excel
email	text		Excel
oldEmail	text		previous email when a newer file changed it
legacyId	text		old CRM id (the club CSV's oldId)
mobile	text		Excel · 05X-XXXXXXX
phone	text		Excel · landline
town	text		Excel · city pie and map
street	text		Excel
houseNo	text		Excel
postalCode	text		Excel
birthday	date		Excel
status	enum		ACTIVE · INACTIVE · ARCHIVED
source	enum		EXCEL · EVENTIM · STORE · APP · MANUAL — where the row was first created
enrichedAt	timestamptz		null = shell fan created by a sync, contacts not yet pulled
emailOptIn	boolean		

smsOptIn	boolean	
whatsappOptIn	boolean	
pushOptIn	boolean	
unsubscribedAt	timestamptz	global marketing stop
engagementScore	int	0–100, computed nightly
appLastLoginAt	timestamptz	copied from AppUser
firstSeenAt	date	"member since": earliest season seat, ticket or order (computed)
lastActivityAt	timestamptz	latest ticket, scan, order or app login (computed)
importedAt	timestamptz	
deletedAt	timestamptz	soft delete
createdAt	timestamptz	
updatedAt	timestamptz	

Unique (clubId, customerId). Indexes on town, email, mobile, engagementScore.

Season		6 columns	–
id	text	PK	
clubId	text	FK → Club	
name	text	UK	"2026/27"
startDate	date		
endDate	date		
isCurrent	boolean		

Unique (clubId, name).

FanSeason — 26 columns

id	text	PK	
fanId	text	FK → Fan	
seasonId	text	FK → Season	
source	enum		both · season details only · seats only
series	text		Excel
tier	enum		REGULAR · SILVER · VIP — mapped from series by a lookup; the מנוי type in every table
seatsCount	int		Excel · > 0 = subscriber
seats	text		Excel · "G10 · row 18 · seat 1; ..."
blocks	text		Excel
seatStatus	text		Excel
seatsTotalPrice	numeric		Excel
currency	text		Excel
orders	text		Excel
invoices	text		Excel
payType	text		Excel
numberOfYears	int		Excel
seasonTickets	int		details: Number of season tickets
seasonTicketSales	numeric		details: Sales volume - season tickets
lastSeasonTicketAt	date		details: Last Sea.Tic. purchase
tickets	int		details: No. of tickets

lastTicketAt	date	details: Last ticket sale
totalSales	numeric	details: Total sales
status	enum	subscriber · new · returned · churned · none — computed vs. previous season
cardNumber	text	Fortress member card, from FullActiveExtract (optional)
details	jsonb	any other details: column, kept as is
importedAt	timestamptz	

Unique (fanId, seasonId). Indexes on (seasonId, status), (seasonId, seatsCount).

Game	21 columns		–
id	text	PK	
clubId	text	FK → Club	
seasonId	text	FK → Season	derived from the date; Eventim has no season
eventimEventId	text	UK	Eventim _ID
title	text		Eventim Title, e.g. הפועל באר שבע נגד ...
venue	enum		HOME · AWAY — from VenueShortCode
venueName	text		Eventim VenueName
venueShortCode	text		
genre	text		Eventim Genre
competition	enum		LEAGUE · CUP · EUROPE · OTHER — set by admin
opponent	text		set by admin or parsed from title
kickoffAt	timestamptz		Eventim Date
status	enum		SCHEDULED · PLAYED · CANCELLED
capacity	int		stadium capacity (Turner 16,126); default per venue, editable
goalsFor	int		result — admin or results feed; Eventim has none
goalsAgainst	int		
fortressProductCode	text		SHF_EVENT_<eventimEventId>
lastFetchTimestampUtc	text		NewTimestampUTC from the last FetchTicketData — the sync cursor
ticketsSold	int		computed from Ticket
attendance	int		computed from Attendance

syncedAt	timestamptz		

Unique (clubId, eventimEventId). Index on (seasonId, kickoffAt).

Attendance — 9 columns

id	text	PK	
fanId	text	FK → Fan	
customerId	text		from the ticket the barcode belongs to
gameId	text	FK → Game	
attended	boolean		null = no scan data for this game yet
barcode	text		Fortress mediaIdentifier = Eventim BC
turnstile	text		Fortress turnstile / gate code
scannedAt	timestamptz		
syncedAt	timestamptz		

Unique (fanId, gameId). Filled from the Fortress scan log once that endpoint is available.

Ticket		19 columns	–
id	text	PK	
fanId	text	FK → Fan	
customerId	text		Eventim CustNr
gameId	text	FK → Game	
barcode	text		Eventim BC; season-ticket barcodes do not end with the event id
orderId	text		Eventim OID — group by it for orders
row	text		Eventim Row
seat	text		Eventim Seat
area	text		Eventim AreaText
entrance	text		Eventim EntryText
price	numeric		Eventim Price
salesType	text		Eventim SalesType (price type code)
salesChannel	text		Eventim MdtNr; 6000 is skipped
status	enum		VALID · CANCELLED — Eventim Status V / S
cancelReason	text		Eventim CReason, e.g. Reprint
bookedAt	timestamptz		Eventim BDate
isSubscription	boolean		Eventim SubscriptionFlag
printAtHome	boolean		Eventim PrintAtHomeFlag
syncedAt	timestamptz		

Unique (gameId, barcode). Indexes on (fanId, gameId), (gameId, status), (orderId).

StoreOrder		24 columns	–
id	text	PK	
clubId	text	FK → Club	
fanId	text	FK → Fan	empty until matched
wooOrderId	int	UK	WooCommerce id — upsert key
orderNumber	text		WooCommerce number (what the club sees in WP admin)
wooCustomerId	int		WooCommerce customer_id; 0 / null = guest checkout
customerNumber	text		order meta _club_customer_number, if the club adds the field
customerName	text		billing first_name + last_name
email	text		billing.email, lower-cased
phone	text		billing.phone, normalized 05X-XXXXXXX
billingCity	text		billing.city — matching hint and geography
matchedBy	enum		CUSTOMER_NUMBER · EMAIL · PHONE · NONE
status	enum		PAID · PENDING · REFUNDED · CANCELLED — from wooStatus
wooStatus	text		raw: completed · processing · on-hold · pending · refunded · cancelled · failed
channel	enum		ONLINE · POS · ADMIN — from created_via
paymentMethod	text		payment_method_title
orderedAt	timestamptz		date_created_gmt
paidAt	timestamptz		date_paid_gmt
modifiedAt	timestamptz		date_modified_gmt — the sync cursor
total	numeric		order total incl. shipping and tax

itemsTotal	numeric	sum of line totals — merchandise revenue
refundTotal	numeric	sum of refunds
currency	text	ILS
syncedAt	timestamptz	

Unique (clubId, wooOrderId). Indexes on (fanId, orderedAt), (clubId, email), (clubId, matchedBy), (clubId, modifiedAt).

StoreOrderItem — 11 columns

id	text	PK	
orderId	text	FK → StoreOrder	
wooLineItemId	int		line_items[].id — upsert key inside the order
productId	int		line_items[].product_id
variationId	int		line_items[].variation_id (size / colour)
sku	text		
name	text		as sold
category	text		first category of the product, looked up from /products at sync
qty	int		quantity
unitPrice	numeric		price
lineTotal	numeric		total after discounts

Unique (orderId, wooLineItemId). Indexes on (productId), (sku). Top products, categories and monthly revenue group by this table.

AppUser — 17 columns

id	text	PK	
clubId	text	FK → Club	
fanId	text	FK → Fan	empty until matched
appUserId	text	UK	the app's user id
customerId	text		if the fan entered it in the app
email	text		
phone	text		
matchedBy	enum		CUSTOMER_ID · EMAIL · PHONE · NONE
platform	enum		IOS · ANDROID
pushToken	text		
pushEnabled	boolean		
lastLoginAt	timestamptz		copied to Fan.appLastLoginAt
logins30d	int		
sessions30d	int		
avgSessionMin	numeric		average session length, last 30 days
lastScreen	text		
syncedAt	timestamptz		

Unique (clubId, appUserId).